

Ilmam

Setup, execution and integration

Status: Proposed work package, not a Jira assignment, client approval, or evidence of completion. Match accounts and agree the task before starting.

Shared goal and how to use this guide

The intended Sprint 1 goal is: sign in, add a supported competitor listing, request collection, and see a trustworthy observation in your own workspace. Development is planned for 28 September–11 October 2026; evaluation starts 12 October (12–18 October window). Verify course updates. No individual completion date or available hours are assumed.

First connect a page to one backend response labelled **synthetic**. This is a short learning/integration checkpoint, not the full sprint or live Amazon acceptance. Accounts, persistence, jobs and permitted source collection follow only when their dependencies are ready.

Read `docs/manual.md` sections 2–5, then the sections named below. Follow one step at a time; the later steps are a route toward the goal, not an instruction to implement the entire guide unattended. Jira owns actual status and assignments; this PDF is a dated handout.

Your deliverable and boundaries

Make the application reproducible, then help deliver reliable collection execution. You own implementation as well as checks; you are not the team's only tester. Reviewer: Hirukshanan; Shewon reviews source execution limits. Relevant scope: PB-03/06/12, FR-05/06/11, NFR-02/03/07–09/11. Read manual sections 5–8 and 10.

Start with the minimum tools for the sample flow. No Kubernetes, cloud account, paid service or extra database/queue system for appearance. The proposed Node/npm/React/Vite/Fastify baseline needs verification, not blind copying of the archived package file.

First working slice: reproducible setup

1. Check Git, runtime, package tool and editor availability on team machines. Record actual blockers; do not assume identical operating systems or hardware. Agree the minimum baseline with the team once.
2. Establish the shared project setup with A/B. Pin a tested runtime expectation and compatible package versions through one lockfile. Provide short commands for starting the page and backend; placeholders must not be described as working commands.
3. Help a second member run from the actual shared source on a clean directory. Before publication, a clean source copy can be a preliminary check; after publication, repeat from a clean clone. Record which was tested.
4. Verify A's page reaches B's synthetic endpoint. Document ports and the local request/proxy path only as implemented. No public exposure of personal development machines.
5. Add one repeatable check command when tests/types exist. Add basic CI after local commands work and remote access is available. Do not report CI success from a local-only run.

Checkpoint evidence: Another member runs the connected sample flow using your instructions, and actual commands/results are recorded. The sample checkpoint does not need PostgreSQL or the job queue yet.

Next slices toward the full sprint goal

1. **Database setup.** With B, trial the proposed PostgreSQL route on team hardware and run migrations. Pick a reproducible native or container path; do not require every possible deployment method. Check no credentials enter Git.
2. **Queue compatibility.** Trial the recommended PostgreSQL-backed queue and separate worker. Explain a persisted job and what happens if the worker stops. Verify library/runtime compatibility before dependent code commits to it.
3. **Authorized enqueue.** With B, accept collection work only for the verified workspace/monitor. Agree atomic or recoverable enqueue and result persistence; do not confuse a saved job with guaranteed crash recovery.
4. **Worker integration.** Call C's adapter through the agreed contract, starting with fixtures. Save attempt status and a valid observation or safe failure. Do not duplicate source parsing or embed source rules into the queue.
5. **Recovery checks.** Use controlled local tests for interruption, duplicate requests/replay, source failure and previous-success preservation. Coordinate bounded retries/concurrency with C; never load-test external sources. Demonstrate that dashboard stored-data reads remain available when collection is unavailable.
6. **Restore and demo.** With B, back up a test database, restore into a separate database and verify relevant records. Rehearse clean setup and the integrated journey with everyone. Document actual limits; local operation is not a 24/7 hosted service.

Handoffs and explain-back

You need B's migration/auth/ownership rules, C's adapter/errors and A's collection-status needs. Provide reproducible commands and runtime/job behaviour early. Each owner still tests their own component.

Before review, explain: What processes run and why? How does the page reach the API? Where does work wait if the worker stops? What prevents duplicates or wrong-workspace results? Which recovery checks did you actually execute?

Review, evidence, and AI use

Before each change, explain its purpose and predict how you will check it. Implement a small change; run the check yourself; explain the result. Ask for help after a focused attempt if blocked. Never guess successful results or hide uncertainty.

Use AI as a tutor: "Read my task and the relevant manual section. Ask me to explain it. Help me plan and implement one small step, then interpret my actual check result. Do not complete the whole assignment or change shared contracts without discussion."

For review, record: task key, what changed, why, command or walkthrough performed, actual result, limitations, material AI help, and reviewer. Use a real Jira key in your branch/PR, not the name of this guide. Done means relevant checks pass, a teammate reviews your explanation and change, and the result is integrated and demonstrated. Research tasks finish with reviewed evidence and a verdict, not invented implementation.

Stop and coordinate if an interface changes, a source requires unsupported access, a paid service is needed, or selected scope is no longer credible. Do not switch frameworks, broaden scope, or claim mock data is live. Use current repository source if this PDF is outdated.